



UNIVERSITY OF
LEICESTER

**School of Computing and
Mathematical Sciences**

CO7201 Individual Project

Final Report - Template

**CulinaAI: An AI-Assisted Recipe
Generation and Meal Personalisation
System**

Sumit Nandu Yadav
sny2@student.le.ac.uk
Student ID – 249021470

**Project Supervisor: Dr Tanya Vladimirova
Principal Marker: Dr Newman Lau**

**Word Count: 4931
14/08/2026**

Abstract

CulinaAI is a software development project that explores how generative AI can support personalised recipe generation and meal planning within a full-stack web application. The system is designed for users who want recipe suggestions based on available ingredients, dietary preferences, allergies, cooking time, budget, equipment and pantry items.

The final report will explain how CulinaAI was designed and implemented using Django, PostgreSQL, Django templates, Bootstrap, JavaScript and the OpenAI API. The main discussion will cover AI recipe generation, constraint-based validation, Smart Pantry, saved recipes, favourites, recipe modification, image generation, comparison tables, AI cooking assistance, cooking mode, dashboard analytics, Django Admin and testing evidence.

This template provides the planned structure of the final dissertation. Each section currently contains a short sketch of the intended content and will be expanded in the final report with literature review, diagrams, source-code excerpts, screenshots, systematic testing, evaluation and critical reflection.

Declaration

All sentences or passages quoted in this report, or computer code of any form whatsoever used and/or submitted at any stages, which are taken from other people's work have been specifically acknowledged by clear citation of the source, specifying author, work, date and page(s). Any part of my own written work, or software coding, which is substantially based upon other people's work, is duly accompanied by clear citation of the source, specifying author, work, date and page(s). I understand that failure to do this amounts to plagiarism and will be considered grounds for failure in this module and the degree examination as a whole.

Name: Sumit Nandu Yadav

Date: 14 August 2026

Table of Contents

Chapter 1 – Introduction	5
1.1 Project Context and Background	5
1.2 Motivation	5
1.3 Problem Statement	5
1.4 Aim and Objectives	5
1.5 Research Questions	6
1.6 Project Scope	6
1.7 Main Contributions	7
Chapter 2 – Background and Literature Review	7
2.1 Computational Recipe Intelligence	7
2.2 Recipe Generation and Personalised Recommendation	7
2.3 Large Language Models in Food and Nutrition Applications	7
2.4 LLM Safety, Guardrails and Food-Specific Risks	7
2.5 Pantry-Based Meal Planning and Food Waste Reduction	8
2.6 Existing Systems and Market Comparison	8
2.7 Literature Summary and Identified Gap	8
Chapter 3 – Requirements and Project Planning	9
3.1 Stakeholder and User Analysis	9
3.2 Development Methodology	9
3.3 Requirements Narrative	9
3.4 Functional Requirements	9
3.5 Non-Functional Requirements	10
3.6 Role-Based and User Workflow	11
3.7 Requirement Evaluation Plan	11
3.8 Risk Analysis and Time Plan	11
Chapter 4 – System Design	12
4.1 Overall Architecture	12
4.2 Frontend and Backend Design	12
4.3 User Workflow Design	12
4.4 Database and Model Design	13
4.5 AI Generation and Validation Design	13
4.6 Recipe Modification Design	13
4.7 Security, Privacy and Data Ownership	13
Chapter 5 – Implementation	13
5.1 Development Environment and Tools	13
5.2 Core Feature Implementation	13
5.3 Frontend and Backend Integration	14
5.4 Current Prototype and Implementation Boundaries	14

5.5 Advanced Technical Challenges	14
5.6 Code Excerpts and Technical Contribution	14
Chapter 6 – Testing	15
6.1 Testing Strategy	15
6.2 Functional and Integration Testing	15
6.3 AI Validation Testing	15
6.4 User Interface and Usability Testing	15
6.5 Test Results Summary	15
Chapter 7 – Evaluation	15
7.1 Evaluation Against Objectives	16
7.2 Evaluation Against Requirements	16
7.3 Evaluation of Technical Achievement	16
7.4 Strengths of the Developed System	16
7.5 Limitations	16
7.6 Comparison with Existing Systems	16
Chapter 8 – Critical Reflection and Future Work	16
8.1 Reflection on Development Process	16
8.2 Changes from the Original Plan	16
8.3 Feedback Incorporated	17
8.4 Lessons Learned	17
8.5 Future Work	17
Chapter 9 – Conclusion	17
References	18
Appendices	18
Appendix B: Supervisor and Principal Marker Feedback Incorporated	18
Appendix C: Use of Generative AI Statement Placeholder	18

Chapter 1 – Introduction

This chapter will introduce CulinaAI as a software development project and explain why personalised, AI-assisted recipe support is a useful problem area. It will describe the project background, motivation, problem statement, aim, objectives, research questions, scope and main contributions before outlining the structure of the final report.

1.1 Project Context and Background

This section will introduce the growing use of AI-assisted tools in everyday decision-making and connect this with meal planning and recipe generation. Many users struggle to decide what to cook when they need to consider available ingredients, allergies, dietary preferences, cooking time, budget and equipment. CulinaAI will be presented as a web-based system that uses AI to generate personalised recipes while also checking whether the generated output satisfies practical and safety-related constraints.

1.2 Motivation

The motivation for this project arises from the limitations of traditional recipe search and general meal-planning tools. Users often need to manually search through recipes, check whether ingredients are available, consider allergy or diet restrictions, and adapt recipes themselves. This can be time-consuming and may still produce unsuitable meal suggestions.

CulinaAI is motivated by the opportunity to create a more complete workflow. Instead of only searching for existing recipes, the system generates personalised recipes, validates them, stores them, allows modification, supports pantry-aware decisions and provides cooking assistance. The final report will explain how this makes the project more advanced than a simple CRUD recipe website.

1.3 Problem Statement

Many recipe platforms support searching, saving or planning meals, but fewer provide a single integrated workflow that combines AI recipe generation, allergy and diet validation, pantry-aware suggestions, recipe modification, validation evidence and cooking support. A recipe generated by an AI model may also contain unsuitable ingredients, hidden allergy conflicts or impractical instructions if it is not checked carefully.

The project therefore addresses the need for a web-based system that can generate personalised recipes while also providing a validation layer and user-facing evidence showing whether the recipe satisfies the user's selected constraints.

1.4 Aim and Objectives

The aim of this project is to design, develop and evaluate CulinaAI, an AI-assisted recipe generation and meal personalisation system built using Django, PostgreSQL and the OpenAI API.

The main objectives of the project are to:

1. Design and develop a secure user-specific web application for recipe generation and meal personalisation.

2. Allow users to generate AI recipes using ingredients, cuisine, diet, allergies, time, budget and equipment constraints.
3. Implement a validation engine that checks generated and modified recipes using a score, risk level and detailed validation breakdown.
4. Provide persistent recipe features such as saved recipes, favourites, history, ratings, feedback and community sharing.
5. Develop Smart Pantry functionality to track pantry items, expiry status and ingredient availability.
6. Support pantry-aware shopping lists and recipe generation from available pantry ingredients.
7. Allow users to modify saved recipes and review the new recipe, image, validation report and comparison table before saving.
8. Provide cooking support through an AI cooking assistant, interactive cooking mode and dashboard analytics.
9. Test and evaluate the system against functional, non-functional and AI-specific requirements.

1.5 Research Questions

The project will be guided by the following questions:

1. How can generative AI be integrated into a web application to support personalised recipe generation?
2. What validation methods can be used to reduce the risk of unsuitable AI-generated recipe output?
3. How can pantry data and expiry tracking improve recipe personalisation and meal planning?
4. How can users be supported in reviewing, modifying and comparing AI-generated recipes?
5. To what extent does CulinaAI meet its functional requirements and demonstrate MSc-level software development achievement?

1.6 Project Scope

The project focuses on developing a working software prototype rather than a deployed commercial product. The scope includes authentication, AI recipe generation, validation evidence, saved recipes, favourites, recipe history, Smart Pantry, shopping lists, recipe modification, modified recipe validation, image generation, AI cooking assistant, cooking mode, dashboard analytics and Django Admin.

The system is not intended to provide professional medical, allergy or nutritional advice. Instead, it provides software-based support and visible validation evidence to help users review AI-generated recipes more carefully.

1.7 Main Contributions

The main contribution of CulinaAI is the development of a complete AI-assisted recipe workflow. The project combines recipe generation, validation, storage, modification, comparison, pantry awareness and cooking support within one Django application.

The final report will argue that the project's technical contribution is not only the use of an AI API, but the surrounding software pipeline: structured prompts, validation scoring, fallback handling, database storage, user-specific filtering, pantry-aware logic, comparison tables, UI presentation and testing evidence.

1.8 Report Structure

This section will briefly describe the remaining chapters. The report follows a software development project structure: background and literature review, requirements and planning, system design, implementation, testing, evaluation, reflection, future work and conclusion.

Chapter 2 – Background and Literature Review

This chapter will provide the academic and technical background for CulinaAI. It will be expanded further in the final report because the final-report template requires a detailed literature survey. The literature review will not simply list sources; it will synthesise research themes and explain how CulinaAI responds to gaps in existing work.

2.1 Computational Recipe Intelligence

This section will review research that treats recipes as structured objects that can be analysed, generated, recommended and evaluated. It will explain that recipes are not only text; they contain ingredients, quantities, steps, constraints, timing, cuisine style and user preferences. CulinaAI connects with this area by generating structured recipe outputs and storing related evidence such as validation scores, risk levels and comparison rows.

2.2 Recipe Generation and Personalised Recommendation

This section will discuss recipe generation and recommender-system literature. Traditional recommender systems often suggest existing items based on user preferences or similarity, while CulinaAI generates new recipe text using user-specific constraints. The final report will compare these approaches and explain why generation requires additional validation.

2.3 Large Language Models in Food and Nutrition Applications

This section will discuss how large language models can produce flexible recipe and meal-planning content. It will also explain the limitation that LLMs may produce confident but incorrect or unsuitable output. CulinaAI therefore uses the OpenAI API as one component in a larger software workflow rather than relying on raw AI output alone.

2.4 LLM Safety, Guardrails and Food-Specific Risks

This section will justify the validation engine. Food-related AI systems need additional care because allergy, diet and unsafe cooking advice can affect user trust and safety. The final report will explain how CulinaAI uses rule-based checks, quality scoring, risk levels and fallback handling to reduce the risk of unsuitable generated recipes.

2.5 Pantry-Based Meal Planning and Food Waste Reduction

This section will connect Smart Pantry to the wider problem of food waste and practical meal planning. Pantry-aware recipe systems can help users make better use of ingredients they already have. CulinaAI supports this through pantry item tracking, expiry status, availability flags, Generate from Pantry and pantry-aware shopping lists.

2.6 Existing Systems and Market Comparison

This section will compare CulinaAI with existing recipe and meal-planning systems. Some systems focus on ingredient-based search, some focus on meal planning, and some provide AI-generated recipe text. CulinaAI will be compared against these systems using features such as AI generation, pantry awareness, validation reports, recipe modification, comparison tables, image generation and cooking assistance.

System type	Typical strength	CulinaAI comparison focus
Ingredient-based recipe search	Finds recipes from available ingredients.	CulinaAI generates new recipes and validates constraints.
Meal-planning applications	Support saving, planning and shopping workflows.	CulinaAI connects planning with AI output, pantry logic and validation evidence.
AI recipe generators	Quickly produce flexible recipe text.	CulinaAI adds validation, storage, modification, comparison, image generation and cooking assistance.
General cooking websites	Provide large recipe collections.	CulinaAI focuses on personalised, user-specific generation and review.

2.7 Literature Summary and Identified Gap

The literature review will conclude by identifying the gap addressed by CulinaAI. Existing systems and research show value in recipe recommendation, generation and meal planning, but the gap is a combined workflow that joins AI generation with validation evidence, pantry-aware decision support, modification traceability and practical cooking assistance. CulinaAI is designed to address this gap as a software development project.

Chapter 3 – Requirements and Project Planning

This chapter will present the project requirements and planning approach. It directly responds to earlier supervisor feedback by presenting requirements as a clear narrative rather than only as a bullet list. It will also explain how each requirement connects to the project aim and how it will be evaluated.

3.1 Stakeholder and User Analysis

This section will identify the main stakeholders and users of CulinaAI. The primary user is a registered person who wants recipe suggestions, meal support and pantry-aware cooking help. The system also includes an administrative role through Django Admin, which supports inspection and management of reference data, user records and recipe-related models.

The final report will explain how user-specific data ownership is important because each user should only see their own saved recipes, pantry items, favourites, feedback and history. This supports both usability and privacy.

3.2 Development Methodology

This section will describe the iterative software development approach used during the project. CulinaAI was developed feature by feature, with regular testing, UI refinement, GitLab commits, supervisor feedback and principal marker feedback. The final report will connect this methodology to the 12-week MSc project timeline.

3.3 Requirements Narrative

The system is built around three core pillars. First, users need a secure personal account so their recipes and pantry data remain separate. Second, users need AI-assisted recipe generation that considers their constraints. Third, users need practical meal-support features such as Smart Pantry, shopping lists, recipe modification, cooking mode and AI assistance.

This narrative will be expanded in the final report before the requirements are presented in table form. This approach avoids presenting requirements as an isolated list and shows why each requirement exists.

3.4 Functional Requirements

The functional requirements will be categorised as essential, recommended and optional/advanced. The final report will evaluate each requirement using screenshots, test cases, database evidence and code references.

ID	Category	Requirement sketch	Final evidence to add
FR1	Essential	Users can register, log in, log out and manage personal data.	Authentication screenshots, tests and view/form references.

FR2	Essential	Users can generate AI recipes from ingredients and constraints.	Generation form, result page and prompt flow.
FR3	Essential	Generated recipes receive validation evidence.	Quality score, validation report and check breakdown.
FR4	Recommended	Users can save, favourite, rate, review and share recipes.	Saved recipe, favourite, feedback and community screenshots.
FR5	Recommended	Smart Pantry supports item storage, availability and expiry tracking.	Pantry screenshots, model evidence and test cases.
FR6	Recommended	Users can generate recipes from pantry ingredients.	Generate from Pantry flow and validation evidence.
FR7	Recommended	Shopping list separates missing items from available pantry items.	Shopping list modal, copy/print tests and pantry match evidence.
FR8	Advanced	Users can modify saved recipes and review changes before saving.	Modified preview, new image, validation report and comparison table.
FR9	Advanced	AI cooking assistant and cooking mode support recipe-specific help.	Chat modal, cooking mode screenshots and workflow tests.
F10	Advanced	Dashboard and Django Admin provide analytics and management evidence.	Dashboard screenshots and admin model screenshots.

3.5 Non-Functional Requirements

This section will cover usability, reliability, security, maintainability, performance and data privacy. CulinaAI will be evaluated on whether the interface is understandable, whether errors are handled clearly, whether user data is scoped correctly, whether the code is

organised into maintainable Django apps and services, and whether the system performs acceptably for the intended prototype use.

3.6 Role-Based and User Workflow

This section will describe the main user workflows in CulinaAI. These include registration and login, recipe generation, viewing validation results, saving recipes, adding favourites, managing pantry items, generating from pantry, creating shopping lists, modifying recipes, using the AI assistant and starting cooking mode.

A workflow diagram will be included in the final report to show how a user request moves through the browser, Django view, service layer, OpenAI API, validation engine, database and final user interface.

3.7 Requirement Evaluation Plan

This section will convert requirements into measurable evaluation criteria. This directly addresses earlier feedback that the evaluation plan needed to be more specific and rigorous.

Requirement area	Evaluation method	Target/evidence
Recipe generation	Generate recipes using varied ingredients, diet, allergy and time inputs.	Valid recipe text, stored record and validation score.
Validation engine	Inspect score, risk level, failed checks and passed checks.	Score of at least 85 where constraints are satisfied.
Allergy/diet safety	Test recipes with allergy and diet restrictions.	No conflicting ingredients in generated output.
Pantry workflow	Add pantry items, change availability and test expiry behaviour.	Expired items excluded and expiring items highlighted.
Modification workflow	Modify saved recipe and inspect preview.	New image, validation report and comparison table shown.
Shopping list	Compare recipe ingredients with pantry items.	Missing and available items separated correctly.
User interface	Complete manual end-to-end workflows.	Screenshots, bug-fix notes and usability comments.

3.8 Risk Analysis and Time Plan

This section will include a refined time plan and risk table. The final report will include a Gantt chart and explain how project risks were managed during development.

This section will include a refined time plan and risk table. The final report will include a Gantt chart and explain how project risks were managed during development.

Risk	Likelihood	Impact	Mitigation
OpenAI API failure or quota issue	Medium	High	Add error handling and avoid relying only on live API calls for testing.
Unsafe or inconsistent AI output	High	High	Use structured prompts, validation score, risk level and fallback handling.
Scope becoming too large	Medium	High	Prioritise essential features and move extra ideas to future work.
Database or migration issues	Medium	Medium	Commit migrations regularly and inspect records through Django Admin.
UI complexity	Medium	Medium	Use Bootstrap, reusable styling and iterative visual testing.
Insufficient report evidence	Medium	High	Collect screenshots, testing tables, code excerpts and GitLab evidence.

Chapter 4 – System Design

This chapter will explain how CulinaAI is organised before discussing implementation details. The final report will include diagrams such as architecture diagram, user workflow diagram, class/model diagram, ER diagram and AI validation sequence diagram.

4.1 Overall Architecture

This section will describe the overall Django web architecture. The user interacts with Django templates in the browser. Django views process requests, forms validate user input, service files communicate with the OpenAI API, Django models store records in PostgreSQL, and static CSS/JavaScript improves the user interface.

A high-level architecture diagram will show the flow between user, browser, Django server, PostgreSQL database and external AI service.

4.2 Frontend and Backend Design

This section will explain how the frontend and backend work together. The frontend is built using Django templates, Bootstrap, custom CSS and JavaScript. The backend is built using Django views, forms, models and service modules. The final report will explain how pages such as recipe generation, saved recipe detail, Smart Pantry, dashboard and cooking assistant connect to backend logic.

4.3 User Workflow Design

This section will outline the main workflows from the user's perspective. For example, the recipe generation workflow begins with form input, continues through AI generation and validation, and ends with a formatted recipe result and validation report. The recipe modification workflow begins with a saved recipe, applies a modification prompt, generates a new image, validates the modified output and displays a comparison table.

4.4 Database and Model Design

This section will describe the main Django models used in the project, including Recipe, RecipeHistory, FavouriteRecipe, RecipeRating, RecipeFeedback, PantryItem, Cuisine, MealType, DietPreference and related cooking assistant models. The final report will include an ER or model diagram showing how these records are connected to the authenticated user.

4.5 AI Generation and Validation Design

This section will describe the AI pipeline. User preferences are collected from forms, converted into a structured prompt, sent to the OpenAI API, validated using CulinaAI's validation engine and then displayed with user-facing evidence. The validation design includes score, status, risk level, validation badge, attempt history and detailed check breakdown.

4.6 Recipe Modification Design

This section will explain the design of the recipe modification feature. A saved recipe can be modified using selected change types or custom instructions. CulinaAI generates a new modified recipe, creates a new AI image, validates the modified recipe and shows an original-versus-modified comparison table before saving.

4.7 Security, Privacy and Data Ownership

This section will explain how user-specific records are protected through Django authentication and filtered database queries. It will also discuss the use of environment variables for API keys and email credentials. The final report will acknowledge that the project is a prototype and not a fully deployed production system.

Chapter 5 – Implementation

This chapter will later become one of the most detailed chapters in the final report. In this template, it gives a concise map of the implementation areas and identifies the evidence that will be added.

5.1 Development Environment and Tools

This section will list and justify the development stack: Python, Django, PostgreSQL, Django templates, Bootstrap, custom CSS, JavaScript, OpenAI API, GitLab and Django Admin. The final report will explain why Django was appropriate for fast full-stack development and why PostgreSQL was used for relational data storage.

5.2 Core Feature Implementation

This section will summarise the implemented features in grouped form. These include authentication, AI recipe generation, recipe validation, saved recipes, favourites, history, feedback, Smart Pantry, Generate from Pantry, shopping list, recipe modification, image generation, comparison table, AI cooking assistant, cooking mode, dashboard, community recipes and Django Admin.

5.3 Frontend and Backend Integration

This section will explain how frontend templates and backend views work together. For example, recipe forms collect input, views process the request, service modules call AI or validation logic, models store the result, and templates display formatted output. JavaScript is used for interactive UI behaviour such as recipe formatting, shopping-list copy/print and cooking assistant chat.

5.4 Current Prototype and Implementation Boundaries

This section will clearly distinguish completed functionality from future work. This is important because the final report should accurately evaluate the submitted prototype rather than presenting planned features as completed work. Completed work includes the main AI generation, validation, pantry, modification and cooking-support workflows. Future improvements may include deployment, larger user evaluation and stronger nutrition datasets.

5.5 Advanced Technical Challenges

This section will distinguish technical challenge from general difficulty. The final report will focus on non-obvious problems such as controlling LLM output, checking hidden allergy conflicts, deciding what happens when validation fails, connecting pantry expiry to generation, validating modified recipes, generating a new image for modified outputs and keeping the cooking assistant recipe-specific.

Each challenge will be explained using the structure: problem, why it was non-trivial, alternative approaches, chosen solution, implementation evidence and testing evidence.

5.6 Code Excerpts and Technical Contribution

The final report must include at least three source-code excerpts. The table below identifies the planned excerpts and the technical contribution each one will demonstrate.

Code excerpt	Planned file path	Why it matters
Recipe generation and validation flow	backend/recipes/views.py	Shows how user input, AI generation and validation evidence are connected.
Validation engine checks	backend/recipes/recipe_quality_engine.py	Shows scoring, risk levels and constraint checks.
Recipe modification and comparison	backend/recipes/ai_service.py and backend/recipes/views.py	Shows how modified recipes are generated, validated, imaged and compared.
Pantry-aware shopping logic	backend/recipes/shopping_service.py	Shows how pantry data is compared with recipe ingredients.

Cooking assistant logic	backend/recipes/cooking_chat_service.py	Shows recipe-specific AI assistance and guardrailed interaction.
-------------------------	---	--

Chapter 6 – Testing

This chapter will present testing evidence for the final software. The final report will include automated tests, manual workflow tests, screenshots, validation results, bug-fix evidence and usability observations.

6.1 Testing Strategy

The testing strategy will combine Django tests for backend logic with manual end-to-end testing for user workflows. AI-dependent behaviour will be tested using controlled inputs and validation evidence rather than relying only on subjective judgement.

6.2 Functional and Integration Testing

This section will test whether major workflows operate correctly together. These workflows include registration, login, recipe generation, validation display, saving, favourites, pantry management, shopping list generation, recipe modification, feedback, community sharing, dashboard and admin inspection.

6.3 AI Validation Testing

This section will focus on allergy, diet, cooking time, equipment and ingredient-match tests. The final report will show how validation scores and messages provide evidence that AI output has been checked systematically.

6.4 User Interface and Usability Testing

This section will test whether the interface is clear, responsive and usable. It will include screenshots showing the final UI and notes on improvements made after feedback, such as the full-width modified recipe preview, modified recipe image, validation report and comparison table.

6.5 Test Results Summary

This section will summarise the final test outcomes in table form. Each major requirement will be marked as passed, partially passed or future work, with evidence such as screenshots, database records, validation reports and code references.

Chapter 7 – Evaluation

This chapter will judge how well CulinaAI satisfies the project objectives and demonstrates MSc-level technical achievement. It will connect implementation evidence with the original aim and requirements.

7.1 Evaluation Against Objectives

This section will return to the objectives from Chapter 1 and evaluate whether each one was achieved. The final report will explain which objectives were fully completed, which were partially completed and which are better treated as future work.

7.2 Evaluation Against Requirements

This section will evaluate the essential, recommended and optional/advanced requirements. Evidence will include screenshots, test results, database records, validation reports, code excerpts and GitLab references.

7.3 Evaluation of Technical Achievement

This section will argue that CulinaAI is more than a standard CRUD application. The final report will highlight AI integration, validation scoring, pantry-aware logic, recipe modification, image generation, user-specific data design, dashboard analytics and systematic testing.

7.4 Strengths of the Developed System

This section will discuss the strongest parts of CulinaAI. These include the complete recipe generation workflow, visible validation evidence, Smart Pantry expiry tracking, pantry-aware shopping list, modified recipe comparison, AI cooking assistant, cooking mode, dashboard and use of Django Admin for inspection and management.

7.5 Limitations

This section will honestly discuss limitations. These may include dependence on external APIs, limited formal user evaluation, possible inconsistency of AI output, non-medical status of validation, image-generation limitations and lack of production deployment.

7.6 Comparison with Existing Systems

This section will compare CulinaAI with existing systems discussed in the literature review. The comparison will focus on how CulinaAI combines several capabilities in one pipeline: generation, validation, pantry awareness, modification, comparison and cooking assistance.

Chapter 8 – Critical Reflection and Future Work

This chapter will reflect on the development process, feedback received, project changes and future extensions. It will show how the project evolved from the initial plan into the final software system.

8.1 Reflection on Development Process

This section will explain how the project was developed iteratively. It will discuss how features were prioritised, how testing and UI refinement were carried out and how the project was managed within the available MSc project timeframe.

8.2 Changes from the Original Plan

This section will explain changes made during the project. Some features were expanded, such as validation evidence and recipe modification, because they improved the technical depth of the system. Other ideas may be moved to future work if they were outside the realistic project scope.

8.3 Feedback Incorporated

This section will explicitly show how feedback improved the project. Supervisor feedback will be linked to requirements narrative, technical challenges, evaluation plan, diagrams, risk planning and literature synthesis. Principal marker feedback will be linked to improvements in the recipe modification feature, including full-width preview, new image generation, comparison table and modified recipe validation report.

8.4 Lessons Learned

This section will reflect on lessons learned during the project. These may include the importance of defining measurable requirements early, separating technical challenges from general difficulty, collecting testing evidence throughout development, and designing AI features with validation rather than trusting generated output directly.

8.5 Future Work

Future development may include:

- larger user evaluation with structured usability metrics
- production deployment with secure hosting and monitoring
- stronger nutrition estimation using verified food composition datasets
- barcode, receipt or fridge-scanning improvements for Smart Pantry
- enhanced food-safety validation using hybrid rule-based and learned guardrails
- mobile-responsive improvements and possible mobile app support
- more advanced dashboard analytics and long-term meal-planning features
- improved community recipe discovery and recommendation features

Chapter 9 – Conclusion

This template establishes the planned final report structure for CulinaAI. It keeps all major chapters and subsections while using short sketches to show what each part will contain. The final dissertation will expand this structure with completed literature synthesis, diagrams, implementation details, source-code excerpts, screenshots, testing results, evaluation and reflection.

The final conclusion will summarise the project problem, the developed solution, the main technical contributions and the extent to which CulinaAI achieved its objectives as an MSc-level software development project.

References

The final report will follow a consistent academic referencing format, using scholarly sources, relevant technical documentation, and other reliable references throughout.

Appendices

The appendices will store supporting material that is useful but too detailed for the main report. This will include screenshots, diagrams, code locations, testing tables and reflective notes.

Appendix A: Planned Diagrams and Screenshots

The final report will include planned diagrams and screenshots such as:

- overall architecture diagram
- ER/model diagram
- AI generation and validation workflow
- recipe modification workflow
- Smart Pantry workflow
- screenshots for homepage, login, recipe generation, validation report, saved recipes, favourites, Smart Pantry, shopping list, modification preview, AI assistant, cooking mode, dashboard and Django Admin

Appendix B: Supervisor and Principal Marker Feedback Incorporated

This appendix will summarise how feedback was addressed. Supervisor feedback will be linked to improved requirements narrative, technical challenge explanation, evaluation plan, diagrams, risk planning and literature synthesis. Principal marker feedback will be linked to the improved recipe modification feature.

Appendix C: Use of Generative AI Statement Placeholder

[Placeholder: Inserting the final university-approved statement here if required. This should accurately describe any permitted use of AI tools for formatting, planning or language support, and should follow module guidance.]